

Object Detection

Hrachya Asatryan

The Three Levels of Spatial Vision

- **Classification:** "What is in this image?" (Output: Class Probabilities).
- **Localization:** "Where is the single main object?" (Output: Class + 4 Bounding Box Coordinates: x, y, w, h).
- **Object Detection:** "What and where are ALL the objects?" (Output: Multiple bounding boxes and classes simultaneously).
- *The Engineering Challenge:* A CNN outputs a fixed-size tensor. How do we predict a *variable* number of objects (0 to 100+) in a single forward pass?

Illustration

Classification



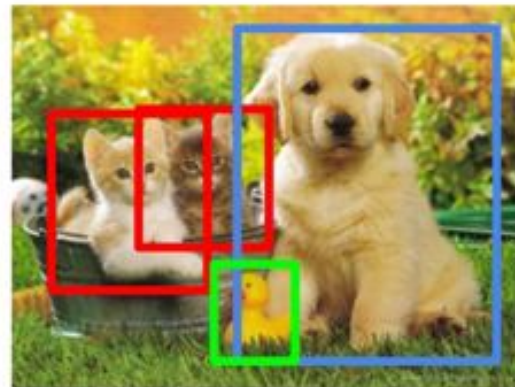
CAT

**Classification
+ Localization**



CAT

Object Detection

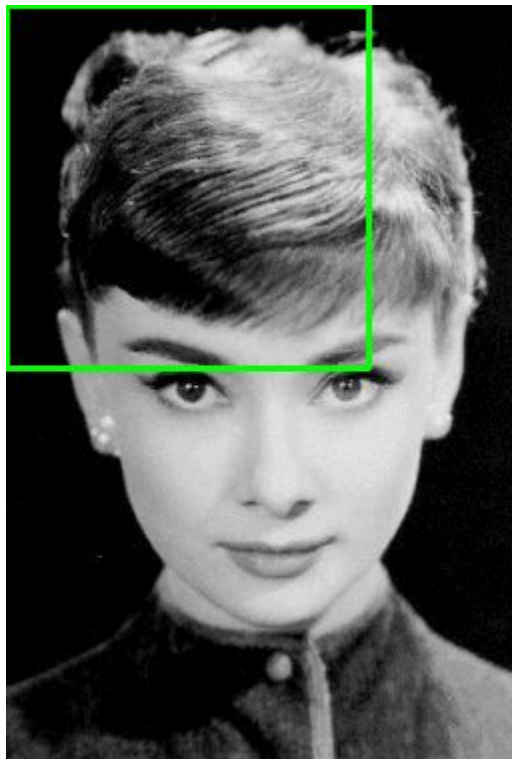


CAT, DOG, DUCK

Sliding Windows (The Dark Ages)

- **Idea:** Take a CNN trained for basic classification. Crop a small window from the top-left of the image. Pass it through the CNN. Move the window slightly to the right. Repeat across the whole image.
- **The Problem:** Objects come in different sizes and aspect ratios. You would need to slide hundreds of thousands of boxes of different sizes across the image.
- *Result:* Computationally impossible for deep networks. We need a smarter way to find "regions of interest."

Illustration



The Two-Stage Era (Accuracy over Speed)

Selective Search

- **The Problem:** We can't evaluate 100,000 windows per image. We need a way to propose a smaller set of "likely" regions (around 2,000) that might contain an object.
- **The Concept:** Selective Search combines the best of two worlds:
 - **Exhaustive Search:** It aims to capture all object scales (small to large).
 - **Image Segmentation:** It uses the structure of the image (pixels that look similar) to guide the search, rather than just guessing random boxes.
- **The Result:** Instead of 100,000 blind guesses, we get 2,000 highly probable "Region Proposals."

Selective Search

- **Step 1: Over-segmentation.** The algorithm starts by breaking the image into thousands of tiny, highly uniform regions based on pixel intensity (usually using Felzenszwalb's algorithm).
- **Step 2: Greedy Merging.** It looks at all neighboring regions and calculates a "Similarity Score." The two most similar regions are merged into a larger region.
- **Step 3: Bounding Boxes.** It draws a bounding box around every single region generated during the merging process.
- **Step 4: Repeat.** It repeats the merging process iteratively until the entire image becomes a single region.

Selective Search

To ensure we capture different types of objects, Selective Search merges regions based on 4 distinct metrics:

1. **Color Similarity:** Do the regions share the same color histograms? (Great for finding a red car).
2. **Texture Similarity:** Do they have similar gradients/edges? (Great for finding a chameleon on a branch).
3. **Size Similarity:** Encourages smaller regions to merge early, preventing a single massive region from swallowing the whole image too fast.
4. **Fill / Shape Compatibility:** Does Region A fit neatly into the gaps of Region B?

The Final Score: A weighted sum of these four metrics determines which regions merge next.

Selective Search



Input Image



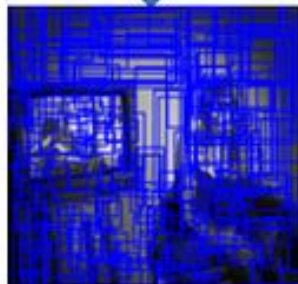
After Initial
Segmentation



After few
Iterations



After many
iterations

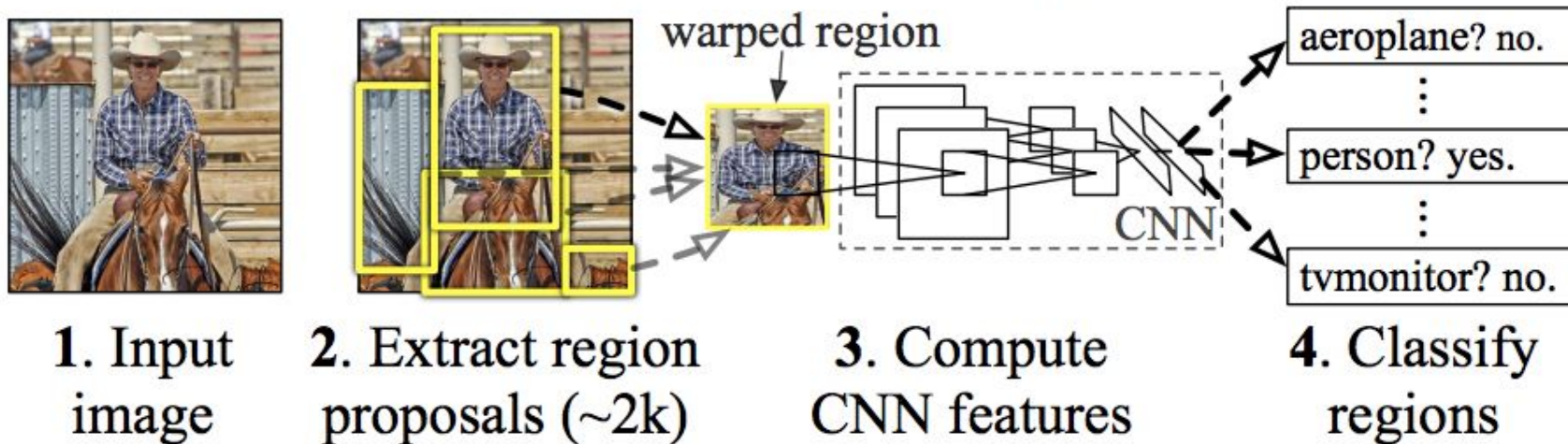


R-CNN (Region-based CNN)

- **Step 1:** Use a traditional CV algorithm (*Selective Search*) to guess ~2,000 regions that *might* contain an object based on color and texture boundaries.
- **Step 2:** Crop and resize each of these 2,000 regions to a fixed size (224x224).
- **Step 3:** Pass all 2,000 crops individually through a CNN (like AlexNet) to extract features.
- **Step 4:** Classify each crop.
- *The Bottleneck:* Running a massive CNN 2,000 times per image took ~45 seconds. Accurate, but way too slow.

R-CNN (Region-based CNN)

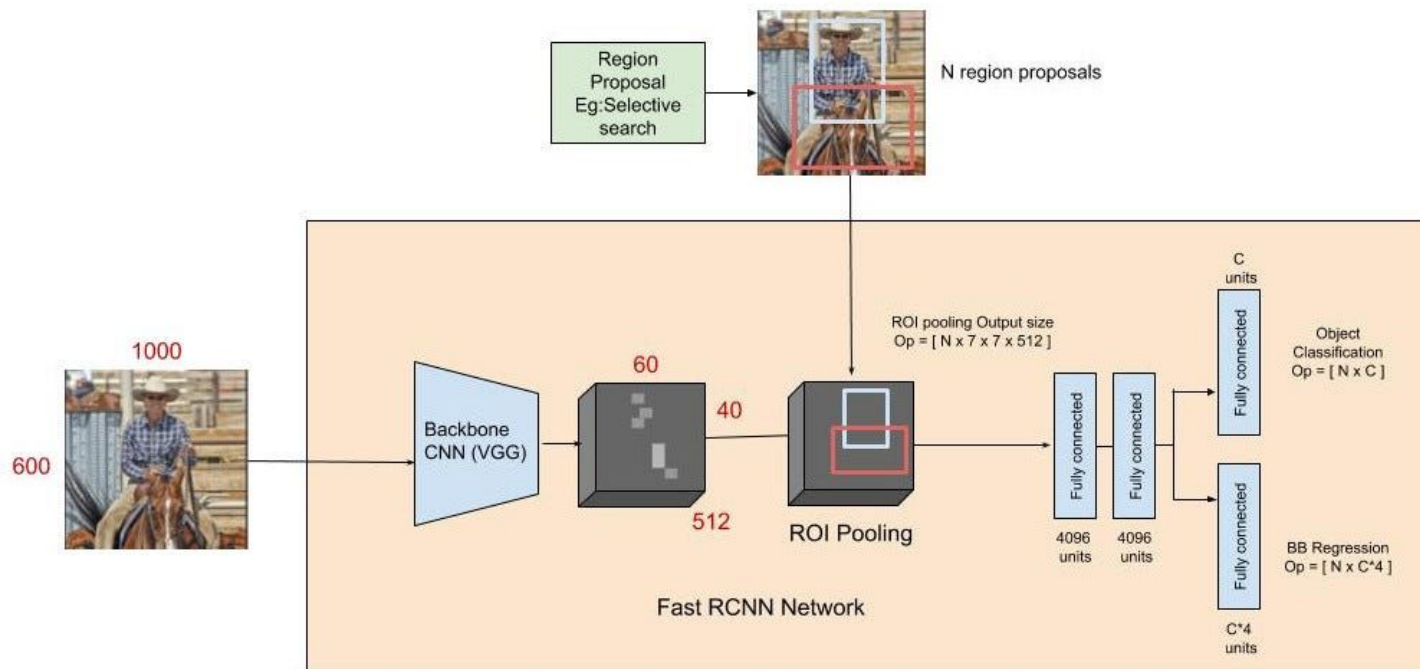
R-CNN: *Regions with CNN features*



Fast R-CNN: Sharing the Computation

- **The Fix:** Don't run the CNN 2,000 times. Run the CNN on the *entire* image *once* to get a giant Feature Map.
- **RoI Pooling:** Take the 2,000 bounding box proposals, and project them onto that Feature Map. Extract the features directly from the map, not the raw image.
- *Result:* Dropped inference time from 45 seconds to ~2 seconds.
- *New Bottleneck:* The Selective Search algorithm itself takes 2 seconds to run. The CNN is now waiting on the CPU.

Fast R-CNN: Sharing the Computation



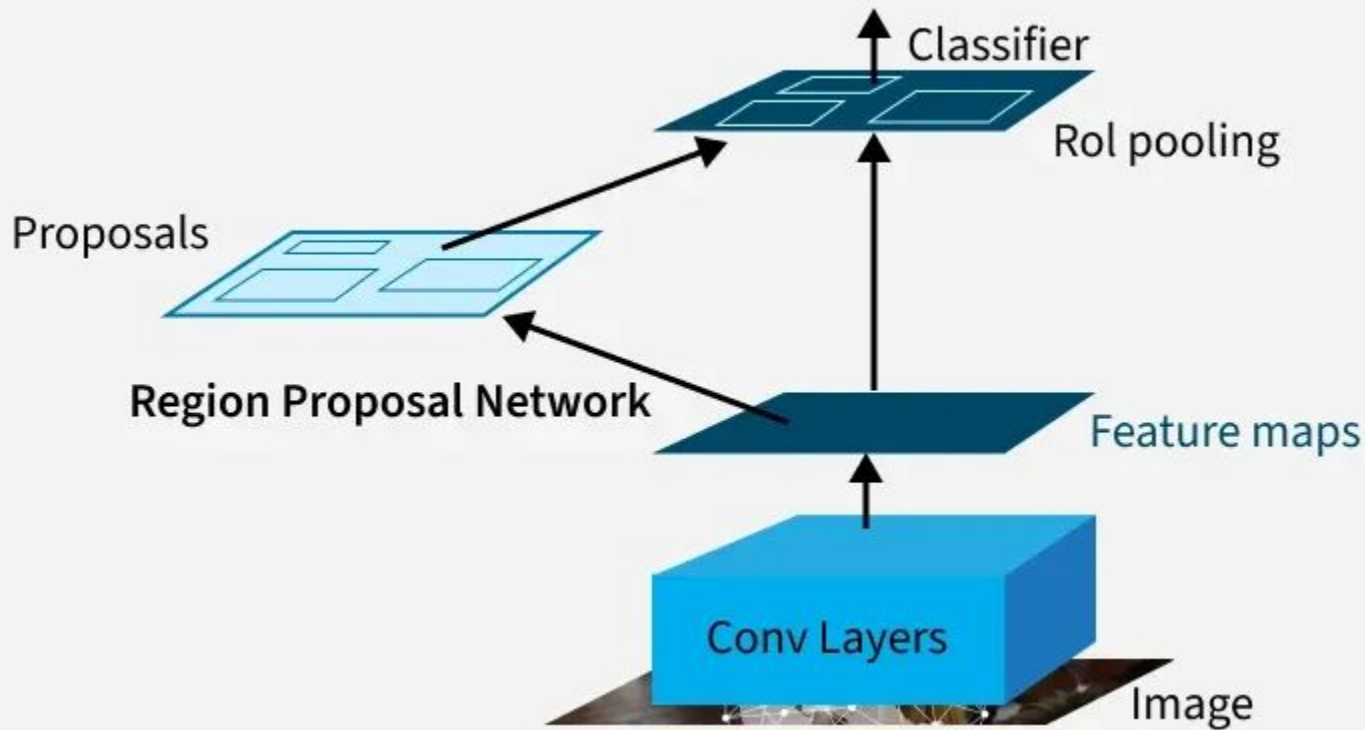
Fast R-CNN: Sharing the Computation

- **The Math Problem:** Neural Network Dense Layers require fixed-size inputs (e.g., 7×7). But our 2,000 proposed boxes are all different shapes and sizes!
- **The Fix: Region of Interest (RoI) Pooling.** No matter the size of the box on the feature map (e.g., 21×14 or 5×9), RoI pooling divides it into a fixed 7×7 grid and performs Max Pooling. This brilliantly forces variable-sized objects into a fixed-size tensor.

Faster R-CNN: Pure Neural Networks

- **The Fix:** Throw away Selective Search entirely. Build a secondary neural network called the **Region Proposal Network (RPN)** to guess the bounding boxes.
- The RPN shares the same convolutional layers as the classifier.
- **The Flow:** Image -> CNN Feature Map -> RPN proposes boxes -> RoI Pooling -> Classifier.
- *Result:* ~0.2 seconds per image (5 FPS). Still the standard for high-accuracy, non-real-time tasks.

Faster R-CNN: Pure Neural Networks



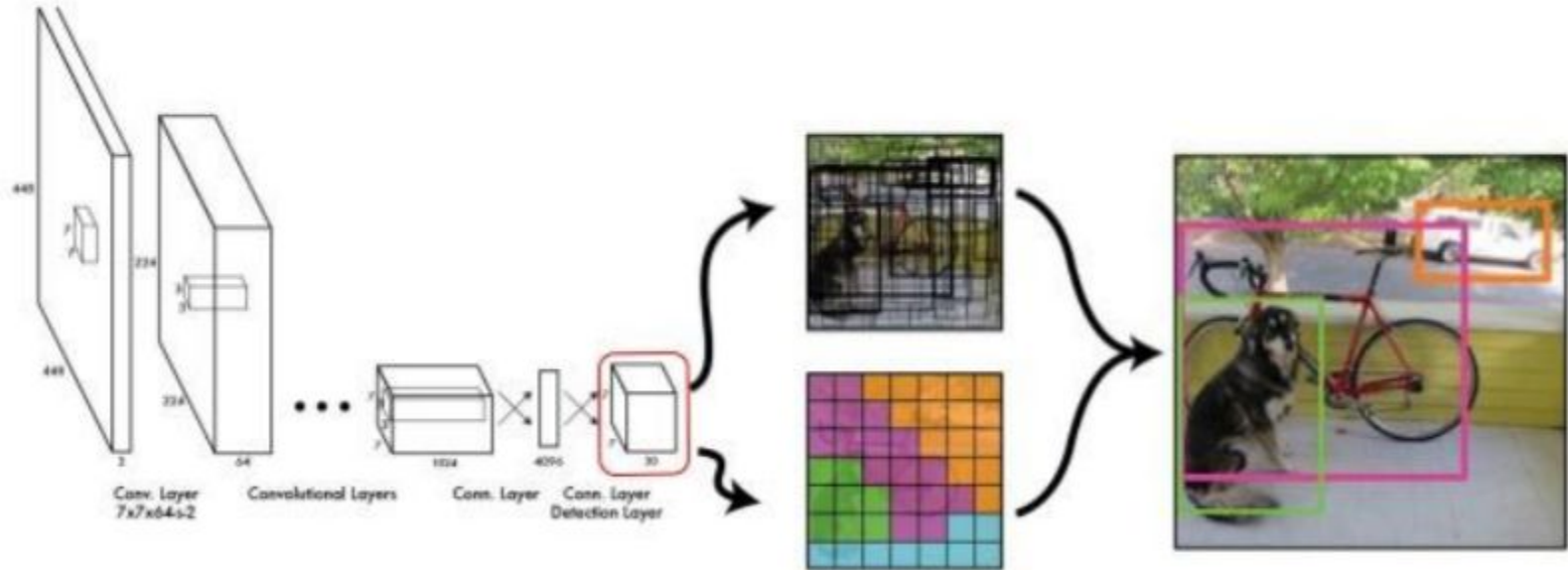
The One-Stage Revolution (Real-Time Vision)

YOLO: You Only Look Once (2016)

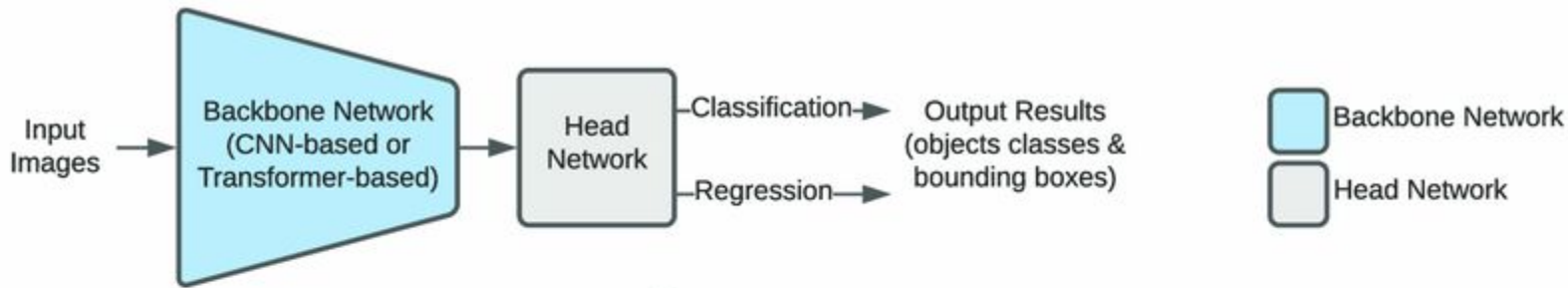
- **The Paradigm Shift:** Stop treating detection as a two-step "Propose Region -> Classify Region" process.
- Treat it as a **single regression problem** from image pixels directly to bounding box coordinates and class probabilities.
- No RPN. No RoI Pooling. Just one single forward pass.
- *Result:* 45 to 150+ FPS. Real-time vision is born.

YOLO: You Only Look Once (2016)

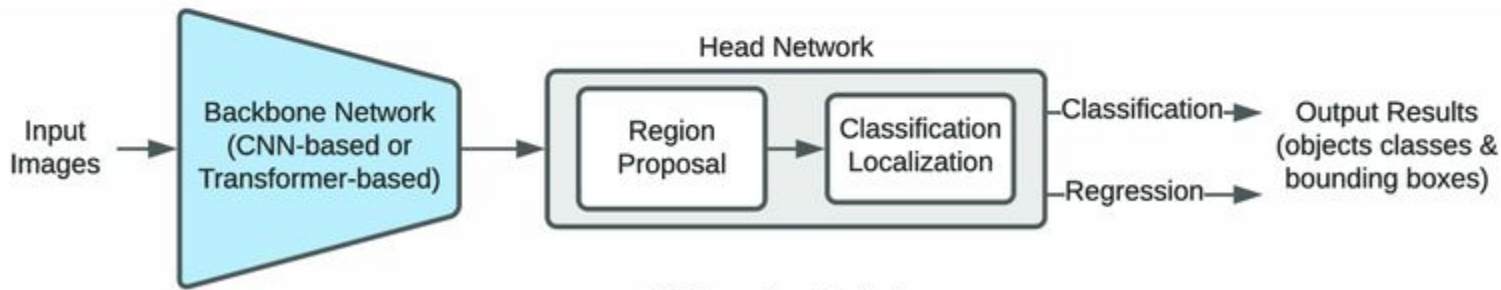
YOLO: You Only Look Once



One vs two stage object detection



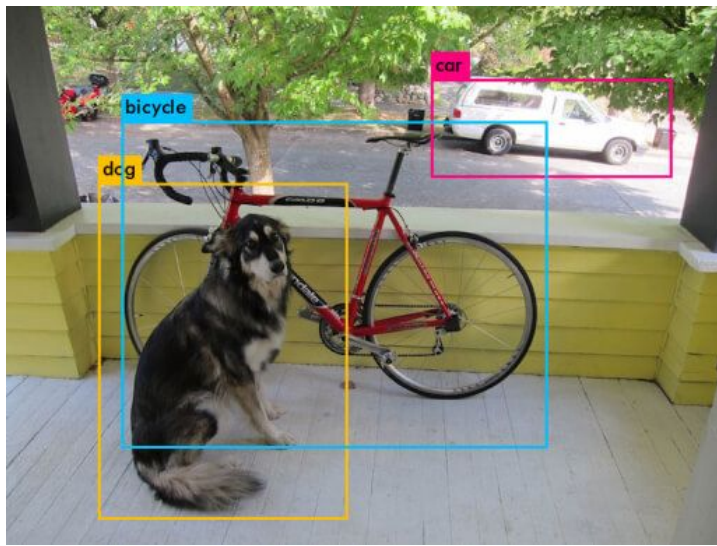
(a) One-stage Detector



(b) Two-stage Detector

How YOLO sees the world

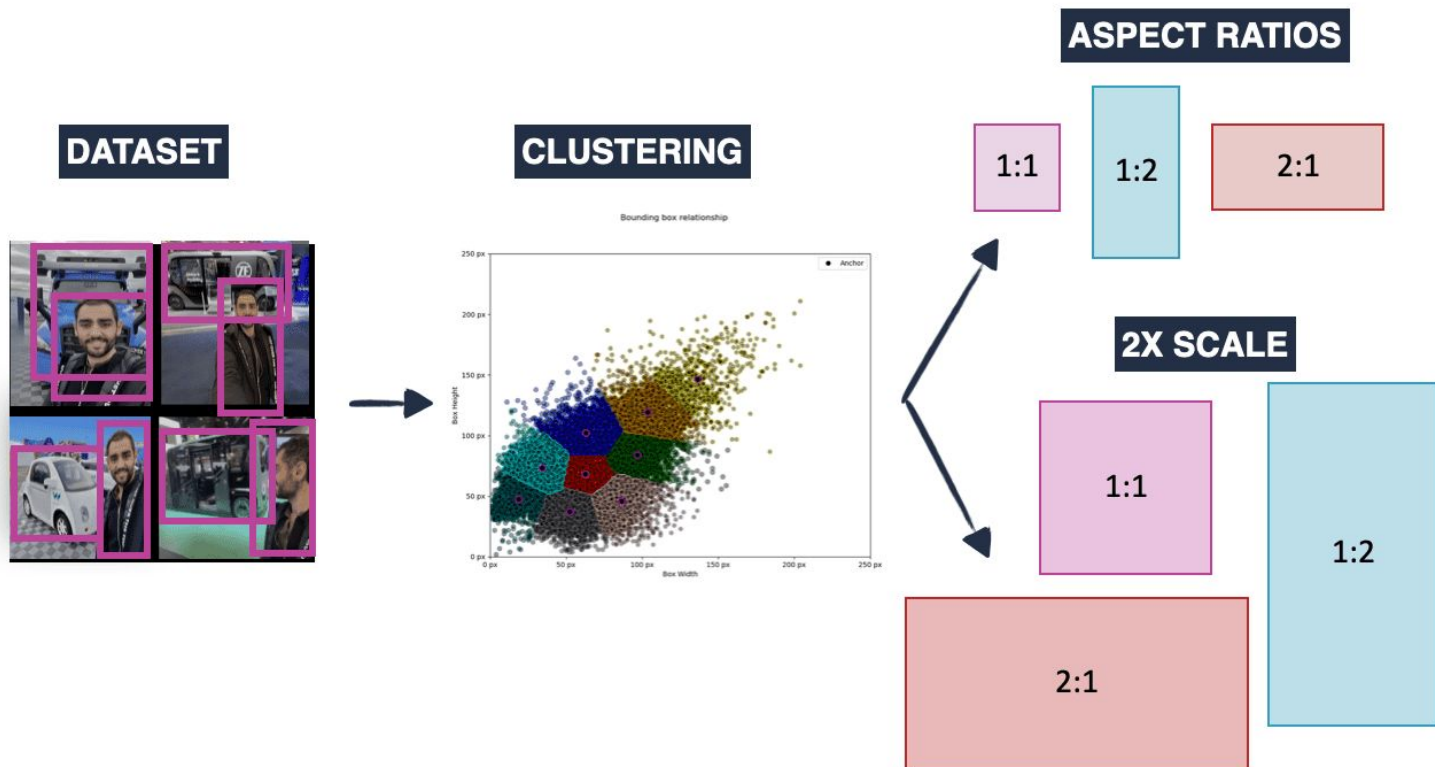
- YOLO divides the input image into an $S \times S$ grid (e.g., 13×13).
- **The Golden Rule:** If the *center* of an object falls into a specific grid cell, *that specific cell* is exclusively responsible for detecting that object.
- Each cell predicts bounding boxes and class probabilities simultaneously.



Anchor Boxes (Priors)

- Asking a neural network to blindly guess bounding box dimensions (x, y, w, h) from scratch is highly unstable.
- Instead, we pre-define standard shapes called **Anchor Boxes** (e.g., a tall thin box for a human, a short wide box for a car).
- YOLO doesn't predict the box directly; it predicts the **offsets** $(\Delta x, \Delta y, \Delta w, \Delta h)$ needed to stretch and move the Anchor Box to fit the object perfectly.

Anchor Boxes (Priors)



The YOLO Output Tensor

- The final layer is a 3D tensor: $S \times S \times (B \times 5 + C)$.
- $S \times S$ The grid (e.g., 13×13).
- B : Number of anchor boxes per cell.
- 5: The coordinates (x, y, w, h) + the **Objectness Score** (Confidence that *any* object is in the box).
- C : Number of classes (e.g., 80 for COCO).

Non-Maximum Suppression (NMS)

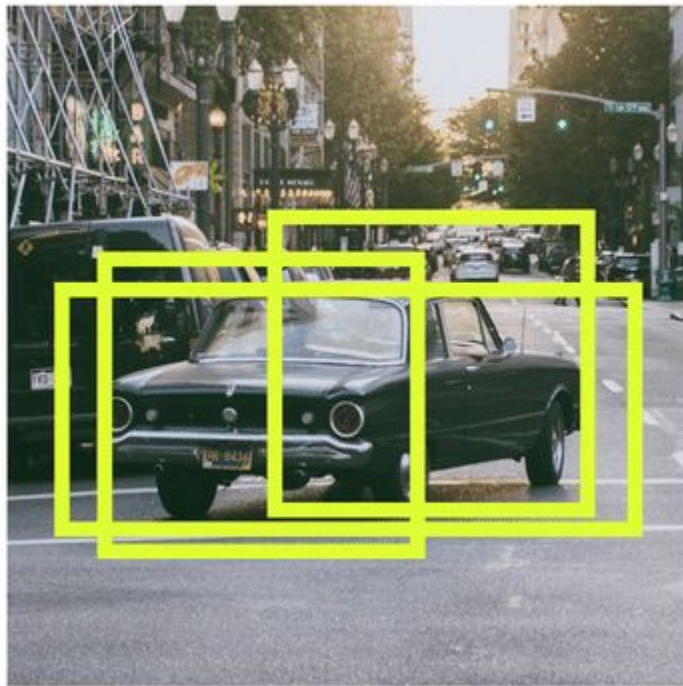
The Problem: Because YOLO predicts thousands of boxes, a single large object (like a car) will trigger detections in multiple adjacent grid cells. You get 5 overlapping boxes for 1 car.

The Solution (NMS):

1. Discard all boxes with an Objectness Confidence < 0.5 .
2. Pick the box with the highest confidence.
3. Calculate the overlap of this box with all other boxes predicting the same class.
4. **Suppress (delete)** any box with too much overlap (they are duplicates).
5. Repeat.

Non-Maximum Suppression (NMS)

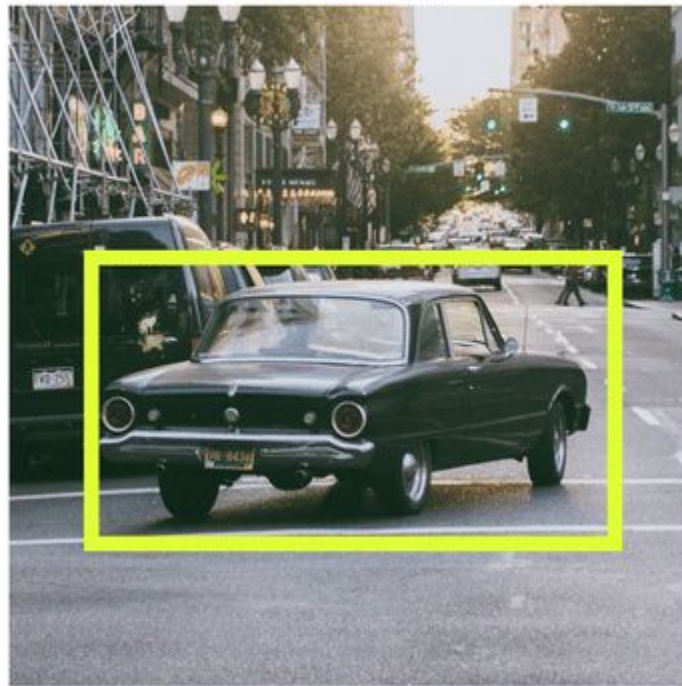
Before non-max suppression



Non-Max
Suppression



After non-max suppression



Loss Function

- **The Problem:** YOLO predicts bounding boxes, confidence scores, and classes all at the exact same time. We can't just use standard Cross-Entropy.
- **The Solution:** YOLO uses a massive **Sum of Squared Errors (SSE)**.
- **The Secret (Responsibility):** YOLO strategically turns parts of the loss function "ON" or "OFF" depending on whether a grid cell is "Responsible" (contains the exact center of a ground truth object).
- **The 4 Components:**
 1. Bounding Box Loss
 2. Confidence Loss (When an object is there)
 3. Confidence Loss (When NO object is there)
 4. Classification Loss

Bounding Box Loss

- **Rule:** This loss is ONLY calculated for the "Responsible" box.
- **Center Error:** Squared difference of the predicted center vs. the true center:

$$(x_{pred} - x_{true})^2 + (y_{pred} - y_{true})^2$$

- **Size Error:** Squared difference of the width and height:

$$(\sqrt{w_{pred}} - \sqrt{w_{true}})^2 + (\sqrt{h_{pred}} - \sqrt{h_{true}})^2$$

- **The Square Root Trick:** Why do we take the square root of the width and height?
 - An error of 10 pixels is a massive deal for a tiny object (like a bird far away).
 - A 10-pixel error means almost nothing for a massive object (like a close-up bus).
 - The square root mathematically forces the loss function to care more about small errors in small boxes!

The Confidence Loss

When an Object IS there (Responsible Cell):

- The true confidence should be 1.0 (or the exact IoU).
- **Loss:** $(C_{\text{pred}} - 1.0)^2$
- *Effect: Heavily penalizes the network if it finds an object but isn't confident about it.*

When NO Object is there (Background Cells):

- The true confidence is 0.0.
- **Loss:** $(C_{\text{pred}} - 0.0)^2$
- **The Imbalance Fix:** Because 90% of an image is empty background, this loss would overwhelm the network, teaching it to just predict "0 confidence" for everything. YOLO multiplies this background loss by a constant ($\lambda_{\text{noobj}} = 0.5$) to make it less powerful.

Classification Loss

Component 4: Classification Loss:

- ONLY calculated for the "Responsible" cell.
- Takes the squared difference between the predicted class probability and the true probability (1.0 for Dog, 0.0 for Cat).
- *Crucially, if a cell does NOT contain an object, YOLO completely ignores its class predictions. Zero penalty.*

The Final Equation: To get the final loss scalar to run `.backward()`, YOLO simply adds them up:

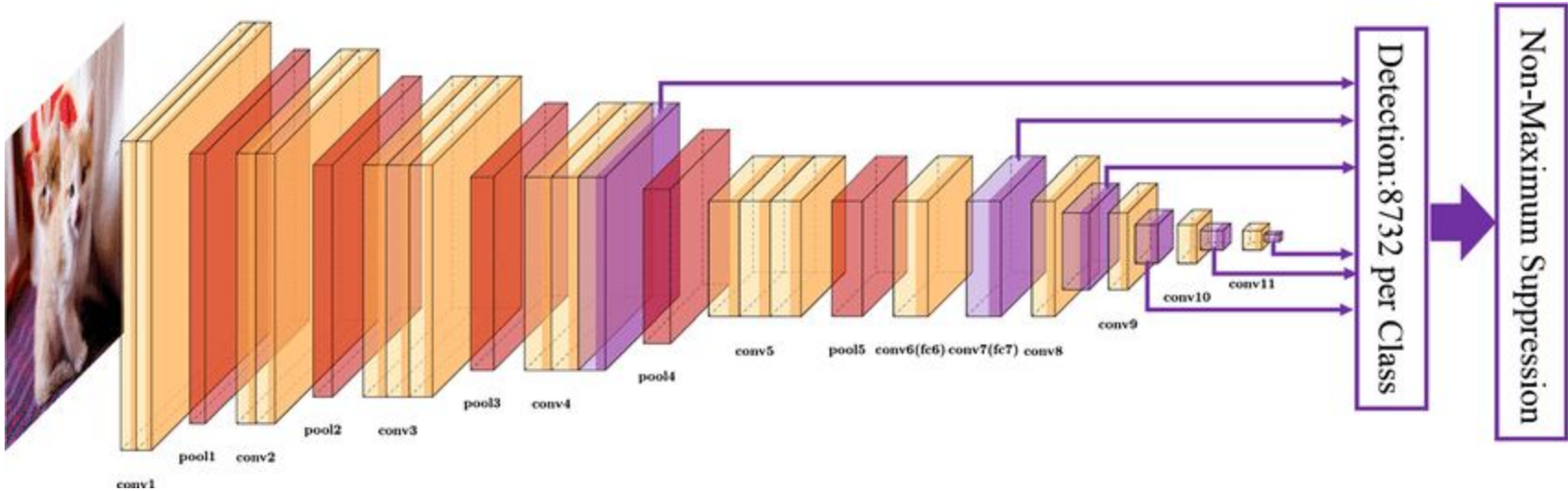
Total Loss = Box Loss + Object Confidence Loss + Background Confidence Loss + Classification Loss

- **Summary:** Empty cells learn to keep their confidence at 0. Responsible cells learn to fine-tune their box shapes and class guesses.

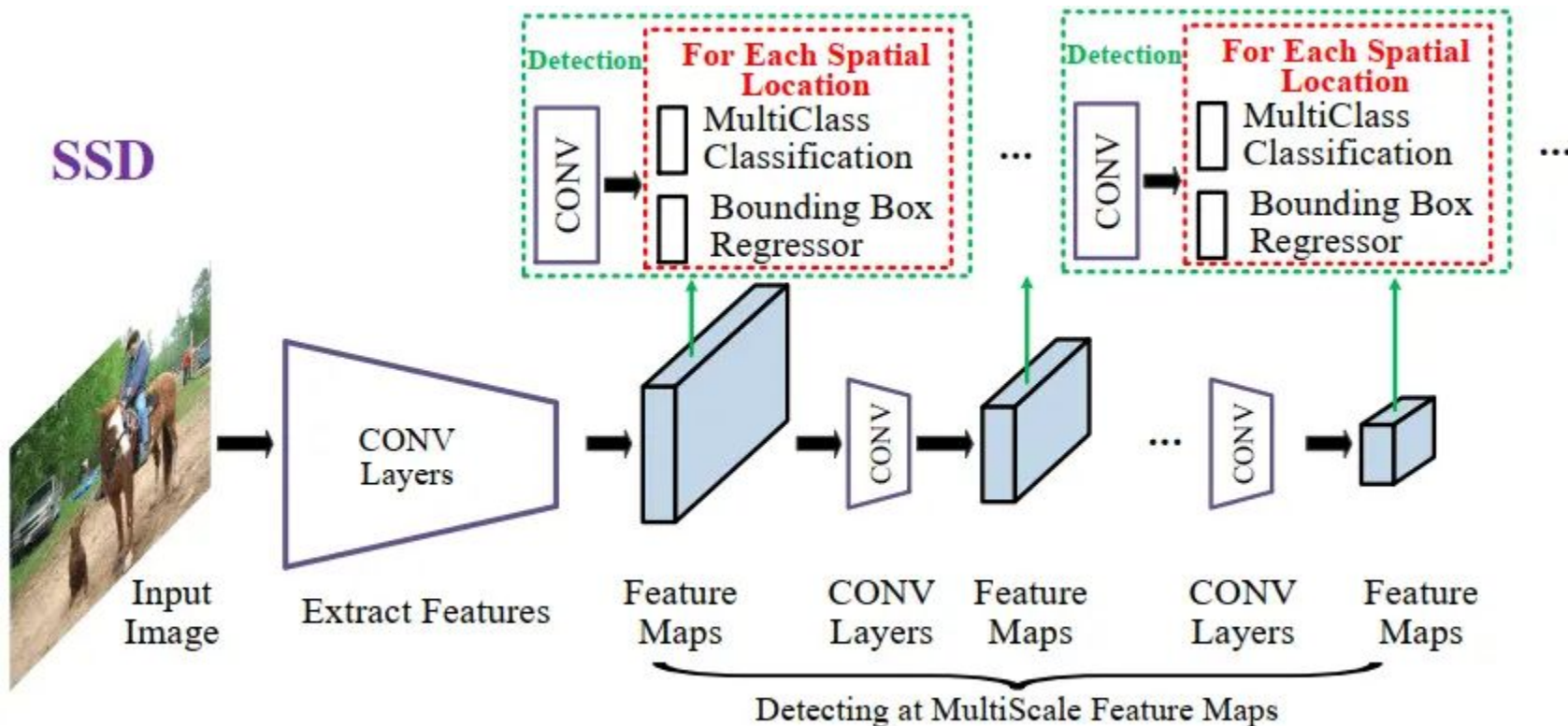
Single Shot MultiBox Detector (SSD)

- **YOLO's Early Weakness:** Struggled with very small objects because a 13x13 grid is too coarse.
- **SSD's Innovation:** Make predictions from *multiple feature maps* at different depths of the network.
- Large feature maps (38x38) detect tiny objects. Small feature maps (5x5) detect huge objects.
- *Note:* Modern YOLO versions have incorporated this multi-scale architecture.

Single Shot MultiBox Detector (SSD)



Single Shot MultiBox Detector (SSD)



Evaluation Metrics

Intersection over Union

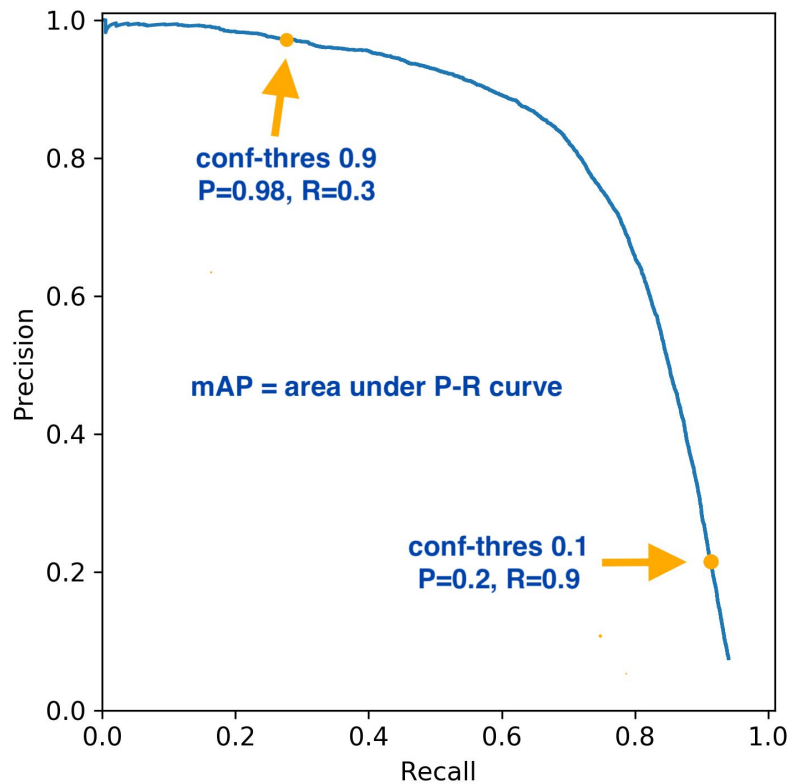
- Now that we can predict boxes, how do we grade them? Accuracy doesn't work well for spatial coordinates.
- **IoU (Jaccard Index) = Area of Overlap / Area of Union.**
- *Thresholding:* We define a detection as a "True Positive" if $\text{IoU} > 0.5$ (or another strict threshold) AND the class label is correct.

Evaluation Metrics

Mean Average Precision (mAP)

- **Precision:** Out of all predicted boxes, how many were right?
- **Recall:** Out of all actual ground-truth objects, how many did we find?
- **Average Precision (AP):** The area under the Precision-Recall curve for a *single class* (e.g., "Dog").
- **mAP:** The mean of the AP across *all* classes.
- (Often calculated across multiple IoU thresholds, written as mAP@0.5:0.95).

Evaluation Metrics



Efficiency

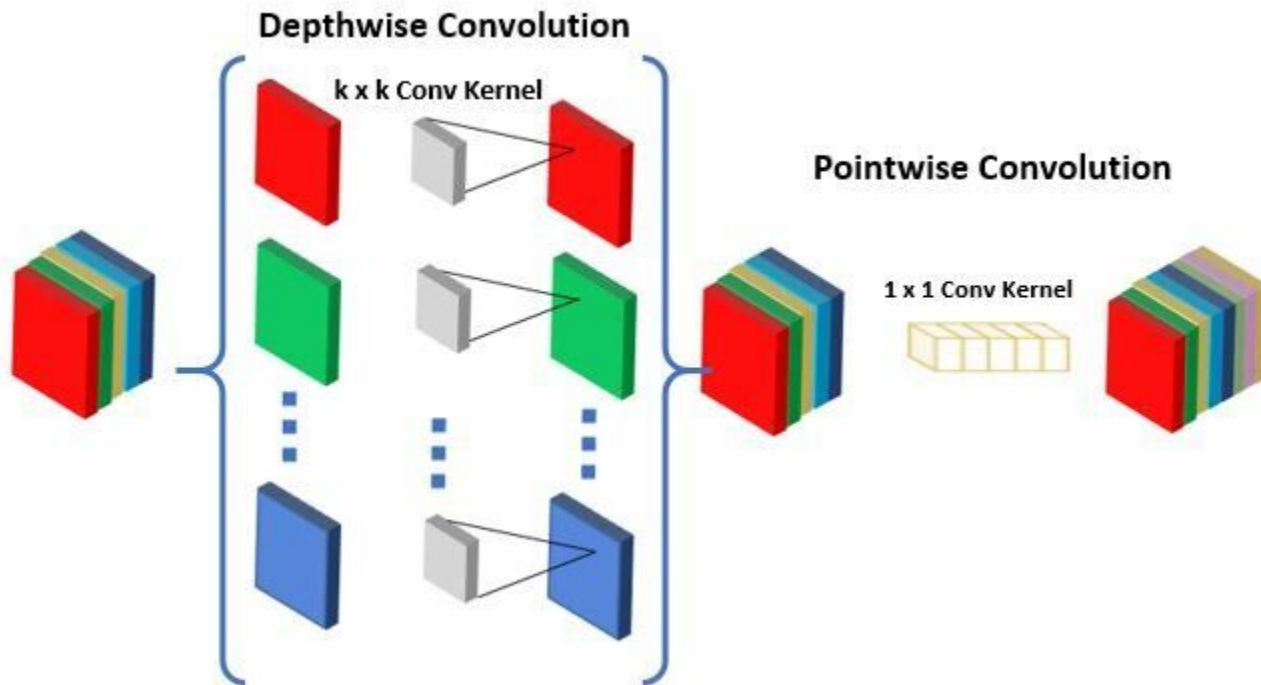
The Deployment Problem

- We have fast algorithms (YOLO) and we can grade them (mAP). But deploying a CNN to a mobile phone or a drone is still mathematically heavy.
- A standard ResNet-50 requires billions of floating-point operations (FLOPs) per image.
- We need to change how the Convolution layer itself works.
- A standard convolution filters spatial dimensions (Height/Width) AND combines channels (RGB/Features) in one single, massive mathematical step.
- Computational Cost scales multiplicatively: $\text{Kernel} \times \text{Kernel} \times \text{Input Channels} \times \text{Output Channels}$.

Depthwise Separable Convolutions

- We split the standard convolution into two distinct, cheaper steps:
- **Step 1: Depthwise Convolution:** Apply a filter to *each channel separately*.
(Spatial filtering only).
- **Step 2: Pointwise Convolution:** Apply a 1x1 convolution across all channels.
(Channel mixing only).
- **The Impact:** For a 3x3 kernel, this reduces the computation by roughly **8 to 9 times** with almost no loss in accuracy. This makes YOLO-Lite and MobileNet possible on phones.

Depthwise Separable Convolutions



Summary

- Two-Stage: Region Proposal → Classify (Accurate, slow).
- One-Stage (YOLO): Grid prediction + Anchor Boxes (Fast).
- NMS: Deletes duplicate boxes.
- Metrics: Use IoU and mAP to measure success.
- Efficiency: Separable convolutions make it all run on edge devices.